Model Training and Evaluation Report

Network Intrusion Detection System — CICIDS2017 Dataset

1. Introduction

This report presents the complete model training and evaluation process for the Network Intrusion Detection System project. The objective is to build a machine learning classifier capable of distinguishing between benign network traffic and various types of cyber attacks using the CICIDS2017 dataset.

The modelling phase follows a structured pipeline: (1) baseline model training across six different algorithms, (2) hyperparameter tuning with integrated class imbalance handling, (3) ensemble model construction, (4) comprehensive performance comparison, error analysis, and final model selection.

2. Experimental Setup

2.1 Dataset

The cleaned CICIDS2017 dataset contains 2,520,762 samples with 52 numerical features and 1 target column (Attack Type). The dataset covers 10 traffic classes:

Class	Samples	Percentage
BENIGN	2,095,057	83.11%
DoS	193,745	7.69%
DDoS	128,014	5.08%
PortScan	90,694	3.60%
Brute Force	9,150	0.36%
Bot	1,948	0.08%

Web Attack – Brute Force	1,470	0.06%
Web Attack – XSS	652	0.03%
Web Attack – SQL Injection	21	0.00%
Heartbleed	11	0.00%

The dataset exhibits severe class imbalance. BENIGN traffic accounts for 83.11% of all samples, while minority attack classes such as Heartbleed (11 samples) and SQL Injection (21 samples) represent less than 0.01% of the data. This imbalance presents a significant challenge for model training and evaluation.

2.2 Data Splitting

The dataset was split into training (80%) and testing (20%) sets using stratified sampling to preserve class proportions. Stratification is critical given the extreme class imbalance, ensuring that even the smallest classes (Heartbleed, SQL Injection) are represented in both sets. The split was performed with a fixed random seed (42) for reproducibility.

Set	**Samples**	**Purpose**
Training	2,016,609	Model fitting and cross-validation
Testing	504,153	Final performance evaluation

2.3 Evaluation Metrics

Given the class imbalance, relying solely on accuracy would be misleading — a model that predicts only BENIGN would achieve 83% accuracy. Therefore, we employ the following metrics:

Metric	**Description**	**Purpose**
F1-Score (weighted)	Weighted average of per-class F1	Overall performance accounting for class frequency
F1-Score (macro)	Unweighted average of per-class	Equal treatment of all classes

	F1	regardless of size
Precision (macro)	Avg precision across classes	False positive rate per class
Recall (macro)	Avg recall across classes	False negative rate per class
ROC-AUC (macro)	Area under ROC curve per class, averaged	Discrimination ability across all classes
Cohen's Kappa	Agreement beyond chance	Model quality vs. random guessing
MCC	Matthews Correlation Coefficient	Balanced measure even with imbalanced classes

F1-macro is used as the primary metric for model selection because it treats all classes equally, making it sensitive to performance on minority attack classes which are the most critical to detect in a security context.

3. Baseline Model Training

Six machine learning algorithms were trained and evaluated as baseline models, ranging from simple linear models to advanced gradient boosting methods. Each model was trained on the full training set using default or commonly recommended hyperparameters.

3.1 Logistic Regression

Logistic Regression serves as a linear baseline. As a linear model, it requires feature scaling (StandardScaler) to ensure convergence. The Limited-memory BFGS (L-BFGS) solver was used with a maximum of 1,000 iterations.

Parameter	**Value**
Solver	L-BFGS
Max iterations	1,000
Feature scaling	StandardScaler
Parallelization	All available cores

Logistic Regression achieved strong overall accuracy (97.62%) but the lowest F1-macro score (0.567), indicating poor performance on minority classes. The model completely failed to detect Web Attack – Brute Force, Web Attack – XSS, and Web Attack – SQL Injection (F1 = 0.000 for all three). This is expected, as linear decision boundaries cannot capture the complex, non-linear patterns differentiating similar attack types.

[INSERT SCREENSHOT: 02_logistic_regression.ipynb — Classification Report & Confusion Matrix]

(From the corresponding notebook output)

3.2 Decision Tree

A Decision Tree classifier was trained with a maximum depth of 20 to prevent excessive overfitting while still capturing complex patterns. Unlike Logistic Regression, Decision Trees do not require feature scaling.

Parameter	Value
Max depth	20
Feature scaling	Not required

The Decision Tree achieved 99.83% accuracy and 0.784 F1-macro. A significant improvement over Logistic Regression, demonstrating the importance of non-linear decision boundaries for this task. The model can detect all attack types to some degree, though minority class performance remains limited (Web Attack – SQL Injection: F1 = 0.400).

[INSERT SCREENSHOT: 03_decision_tree.ipynb — Classification Report, Confusion Matrix & Feature Importance]

(From the corresponding notebook output)

3.3 Random Forest

Random Forest extends the Decision Tree approach by training 300 trees on random subsets of features and samples (bagging), then averaging their predictions. This reduces variance and generally improves generalization.

Parameter	Value
Number of estimators	300
Max depth	None (unlimited)
Min samples split	2
Min samples leaf	1

Random Forest achieved 99.82% accuracy and 0.827 F1-macro. The improvement over a single Decision Tree is notable in macro recall (0.811 vs. 0.751), indicating better detection of minority attack classes. The ensemble effect smooths out individual tree errors.

[INSERT SCREENSHOT: 04_random_forest.ipynb — Classification Report, Confusion Matrix & Feature Importance]

(From the corresponding notebook output)

3.4 XGBoost

XGBoost (Extreme Gradient Boosting) builds trees sequentially, with each tree correcting the errors of the previous one. Sample weights were computed using the 'balanced' strategy to address class imbalance. XGBoost requires label encoding as it operates on numerical targets.

Parameter	Value
Number of estimators	300
Max depth	7
Learning rate	0.1
Subsample	0.8
Colsample by tree	0.8
Imbalance handling	Balanced sample weights

XGBoost achieved the highest F1-macro among all baseline models at 0.8585, along with 99.87% accuracy and a perfect ROC-AUC of 1.0. The combination of gradient boosting and balanced sample weights gave it the best recall on minority classes (macro recall: 0.886). Notably, it achieved F1 = 0.667 on SQL Injection (17 test samples) — the highest among all models.

[INSERT SCREENSHOT: 05_xgboost.ipynb — Classification Report, Confusion Matrix & Feature Importance]

(From the corresponding notebook output)

3.5 LightGBM

LightGBM is a gradient boosting framework that uses histogram-based splitting for faster training. It employs leaf-wise tree growth (rather than level-wise), which can produce deeper, more specialized trees.

Parameter	Value
Number of estimators	200
Number of leaves	31
Learning rate	0.05
Subsample	0.8
Regularization (L1/L2)	0.1 / 0.1

LightGBM achieved the highest weighted F1-score (0.9989) and accuracy (99.89%) among all baseline models, but ranked second in F1-macro (0.830). Its fast training time (69.56s) makes it the most efficient model in the lineup.

[INSERT SCREENSHOT: 06_lightgbm.ipynb — Classification Report, Confusion Matrix & Feature Importance]

(From the corresponding notebook output)

3.6 Extra Trees (Extremely Randomized Trees)

Extra Trees is similar to Random Forest but with an additional source of randomness: instead of searching for the optimal split threshold, it selects thresholds randomly. This further reduces variance at the cost of slightly higher bias.

Parameter	Value
Number of estimators	300
Max depth	None (unlimited)
Min samples split	2
Min samples leaf	1

Extra Trees achieved 99.78% accuracy and 0.813 F1-macro. Performance is comparable to Random Forest, with the randomized splitting providing faster training (81.00s vs. 277.64s) but slightly lower F1-macro.

[INSERT SCREENSHOT: 07_extra_trees.ipynb — Classification Report, Confusion Matrix & Feature Importance]

(From the corresponding notebook output)

3.7 Baseline Comparison Summary

Model	Accuracy	F1 (weighted)	F1 (macro)	ROC-AUC (macro)	MCC	Time (s)
Logistic Regression	0.9762	0.9762	0.567	0.9762	0.9218	113.9
Decision Tree	0.9983	0.9983	0.784	0.9552	0.9944	72.7
Random Forest	0.9982	0.9982	0.827	0.9866	0.9939	277.6
XGBoost	0.9987	0.9987	0.859	1.0000	0.9957	112.8
LightGBM	0.9989	0.9989	0.830	0.9981	0.9962	69.6
Extra Trees	0.9978	0.9978	0.813	0.9963	0.9927	81.0

All tree-based models achieve exceptionally high weighted metrics (>99.7%), reflecting strong overall classification. However, macro metrics reveal significant differences in minority class handling. XGBoost leads with F1-macro of 0.859, followed by LightGBM (0.830) and Random Forest (0.827). Logistic Regression lags significantly (0.567), confirming that this problem requires non-linear methods.

[INSERT SCREENSHOT: 09_model_comparison.ipynb — F1-Macro Bar Chart (STEP 4, left chart)]

(From the corresponding notebook output)

4. Hyperparameter Tuning

4.1 Methodology

Hyperparameter tuning was performed on the top 3 baseline models (Random Forest, XGBoost, LightGBM) using Randomized Search Cross-Validation. The selection of these three models was based on their F1-macro ranking from the baseline evaluation.

Parameter	Value
Search method	RandomizedSearchCV
Number of iterations	30
Cross-validation	3-fold Stratified
Scoring metric	F1-macro
Training subset	20% of training data (403,321 samples)

A 20% stratified subsample of the training data was used for the search phase to reduce computational cost. Class imbalance was addressed through model-specific mechanisms: class_weight='balanced' for Random Forest and LightGBM, and computed sample weights for XGBoost.

4.2 Search Spaces

Random Forest search space:

Parameter	Values Explored
n_estimators	100, 200, 300, 500
max_depth	10, 20, 30, None
min_samples_split	2, 5, 10
min_samples_leaf	1, 2, 4
class_weight	balanced, balanced_subsample, None

XGBoost search space:

Parameter	Values Explored
n_estimators	100, 200, 300, 500
max_depth	3, 5, 7, 10
learning_rate	0.01, 0.05, 0.1, 0.2
subsample	0.6, 0.8, 1.0
colsample_bytree	0.6, 0.8, 1.0
min_child_weight	1, 3, 5

LightGBM search space:

Parameter	Values Explored
n_estimators	100, 200, 300, 500
num_leaves	15, 31, 63, 127
learning_rate	0.01, 0.05, 0.1, 0.2
min_child_samples	5, 10, 20, 50
subsample	0.6, 0.8, 1.0

colsample_bytree	0.6, 0.8, 1.0
class_weight	balanced, None

4.3 Tuning Results

Best hyperparameters found:

Model	Best CV F1-macro	Key Parameters	Tuning Time
Random Forest	0.7095	n_estimators=300, max_depth=None, class_weight=balanced_subsample	2,003s
XGBoost	0.8239	n_estimators=100, max_depth=5, learning_rate=0.01	1,325s
LightGBM	0.7887	n_estimators=500, num_leaves=63, class_weight=balanced	4,174s

4.4 Tuned vs. Baseline Performance

Model	F1-macro (Baseline)	F1-macro (Tuned)	Δ	Verdict
Random Forest	0.827	0.821	-0.006	Comparable
XGBoost	0.859	0.577	-0.282	Degraded
LightGBM	0.830	0.097	-0.733	Collapsed

Unexpectedly, hyperparameter tuning degraded performance for XGBoost and LightGBM, while Random Forest remained approximately unchanged. This counter-intuitive result warrants detailed analysis.

4.5 Analysis: Why Tuning Degraded Performance

The performance degradation can be attributed to the following factors:

1. Extreme minority class noise in cross-validation: The 20% stratified subsample contained only 2 Heartbleed samples and 4 SQL Injection samples. With 3-fold CV, each fold could contain 0 or 1 samples from these classes. The F1-macro score — which gives equal weight to all classes — becomes highly unstable when individual class scores fluctuate between 0.0 and 1.0 based on a single sample. This noise caused RandomizedSearchCV to select parameters that happened to score well on these random fluctuations rather than genuinely good configurations.

2. Over-aggressive class rebalancing: When class_weight='balanced' is applied to extremely rare classes (e.g., Heartbleed with 2 samples), the model assigns disproportionately large weights to these samples. This causes the model to over-predict minority classes at the expense of majority class accuracy. LightGBM's collapse to F1-macro 0.097 is a direct consequence: its accuracy dropped to 68% because it misclassified large numbers of BENIGN samples as minority attacks.

3. Suboptimal parameter combinations: XGBoost's best parameters included learning_rate=0.01 with only n_estimators=100. This is an underfitting configuration — with such a low learning rate, the model needs significantly more estimators (≥1,000) to converge. The random search did not explore this specific combination.

Key Takeaway: When dealing with extreme class imbalance (classes with <20 samples), aggressive rebalancing techniques can do more harm than good. The baseline models, which used moderate or no rebalancing, achieved better performance by maintaining a healthy trade-off between majority and minority class detection.

[INSERT SCREENSHOT: 08_hyperparameter_tuning.ipynb — Tuned Model Comparison Table (STEP 9)]

(From the corresponding notebook output)

5. Ensemble Model

To explore whether combining multiple models could improve overall performance, a Soft Voting Classifier was constructed using the three best baseline models: XGBoost, LightGBM, and Random Forest.

5.1 Methodology

Soft Voting works by averaging the predicted probability distributions from each base model. The final prediction is the class with the highest average probability. This approach is more nuanced than hard voting (majority rule) because it leverages the confidence levels of each model.

Component	Configuration
Base model 1	XGBoost (n_estimators=300, max_depth=7)
Base model 2	LightGBM (n_estimators=300, num_leaves=31)
Base model 3	Random Forest (n_estimators=300)
Voting method	Soft (probability averaging)

5.2 Ensemble Results

The Voting Ensemble model was trained on the full training set and evaluated on the test set.

[INSERT SCREENSHOT: 10_ensemble_model.ipynb — Classification Report & Confusion Matrix]

(From the corresponding notebook output)

The ensemble model achieved marginally higher metrics compared to XGBoost alone, validating that combining diverse models can reduce individual model weaknesses. However, the improvement is small in magnitude.

6. Per-Class Performance Analysis

To understand how each model performs on individual attack types, per-class F1 scores were computed across all six baseline models.

Class	LR	DT	RF	XGBoost	LightGBM	ET
BENIGN	0.986	0.999	0.999	0.999	0.999	0.999
Bot	0.045	0.736	0.819	0.803	0.824	0.796

Brute Force	0.856	0.998	0.997	0.999	0.999	0.996
DDoS	0.984	1.000	1.000	1.000	1.000	1.000
DoS	0.957	0.998	0.997	0.999	0.999	0.996
Heartbleed	1.000	0.667	1.000	1.000	1.000	1.000
PortScan	0.843	0.991	0.988	0.994	0.994	0.988
Web Attack – BF	0.000	0.735	0.774	0.752	0.779	0.736
Web Attack – SQL	0.000	0.400	0.333	0.667	0.333	0.333
Web Attack – XSS	0.000	0.318	0.360	0.373	0.369	0.286

[INSERT SCREENSHOT: 09_model_comparison.ipynb — Per-Class F1 Heatmap (STEP 6)]

(From the corresponding notebook output)

Key observations from per-class analysis:

1. High-volume classes (BENIGN, DDoS, DoS, Brute Force): All tree-based models achieve near-perfect F1 scores (>0.99). These classes have sufficient training data for effective learning.

2. Medium-volume classes (PortScan, Bot): Performance ranges from 0.74 to 0.82 F1. Bot detection is particularly challenging due to its low sample count (1,948 total) and traffic patterns that can mimic benign behavior.

3. Low-volume classes (Web Attack types): These are the most challenging. Web Attack – XSS (F1 ~0.3) and Web Attack – SQL Injection (F1 ~0.3–0.7) suffer from extremely limited training data. The high variability in SQL Injection F1 (0.000 to 0.667) reflects the statistical instability of evaluating models on just 4 test samples.

4. Heartbleed: Despite having only 11 total samples (2 test), most models achieve F1 = 1.0. This suggests Heartbleed traffic has a very distinctive pattern that is easy to distinguish, but the result should be interpreted with extreme caution given the tiny sample size.

7. Error Analysis

Detailed error analysis was performed on the best-performing model (XGBoost baseline) to identify systematic misclassification patterns.

7.1 Top Misclassification Pairs

True Class	Predicted As	Count	% of True Class
BENIGN	PortScan	198	0.05%
BENIGN	Bot	186	0.04%
Web Attack – XSS	Web Attack – BF	81	62.31%
BENIGN	DoS	78	0.02%
Web Attack – BF	Web Attack – XSS	65	22.11%

[INSERT SCREENSHOT: 09_model_comparison.ipynb — Top Misclassification Bar Chart (STEP 7)]

(From the corresponding notebook output)

7.2 Analysis of Error Patterns

1. Web Attack cross-confusion: The most striking pattern is the mutual confusion between Web Attack – XSS and Web Attack – Brute Force. 62.3% of XSS attacks are misclassified as Brute Force, and 22.1% of Brute Force are misclassified as XSS. This occurs because both attack types target web applications and share similar network-level features (HTTP traffic patterns, request rates, packet sizes). Distinguishing between them would likely require application-layer features (e.g., payload content) that are not present in the CICIDS2017 feature set.

2. BENIGN false positives: A small number of benign samples are misclassified as PortScan (198), Bot (186), and DoS (78). In absolute terms, these numbers are significant, but relative to the 419,012 benign test samples, they represent a false positive rate of only 0.11%. In a production IDS, this would generate approximately 1 false alert per 900 benign connections — generally an acceptable rate.

3. Low total errors: For XGBoost, the total number of misclassifications across all classes is approximately 700 out of 504,153 test samples, yielding an error rate of 0.13%. The model's errors are concentrated in the extremely rare attack types where training data is insufficient.

8. Model Selection

8.1 Selection Criteria

The final model was selected based on a multi-criteria ranking across four key metrics: F1-macro, F1-weighted, ROC-AUC macro, and MCC. Each model was ranked 1 (best) to 6 (worst) for each metric, and the average rank determined the final ordering.

Model	F1-macro Rank	F1-weighted Rank	ROC-AUC Rank	MCC Rank	Avg Rank
XGBoost	1	2	1	2	1.50
LightGBM	2	1	2	1	1.50
Random Forest	3	4	4	4	3.75
Decision Tree	5	3	6	3	4.25
Extra Trees	4	5	3	5	4.25
Logistic Regression	6	6	5	6	5.75

8.2 Final Decision: XGBoost

XGBoost and LightGBM are tied in average rank (1.50). XGBoost was selected as the final model for the following reasons:

1. Higher F1-macro (0.859 vs. 0.830): XGBoost is better at detecting minority attack classes, which is the primary objective of an intrusion detection system. Missing an attack (false negative) is more dangerous than a false alarm (false positive).

2. Perfect ROC-AUC (1.0): XGBoost demonstrates perfect class separation in probability space, meaning that with appropriate threshold tuning, its detection rates can be further optimized for specific deployment scenarios.

3. Reasonable training time (112.8s): While not the fastest model, XGBoost's training time is practical for periodic retraining in production environments.

8.3 Why Not the Ensemble Model?

Although the Voting Ensemble achieved marginally higher metrics than XGBoost alone, it was not selected as the final model for the following practical reasons:

1. Model complexity: The ensemble contains three separate models (XGBoost + LightGBM + Random Forest), resulting in a significantly larger model file and higher memory requirements.

2. Inference latency: In network intrusion detection, real-time prediction speed is critical. The ensemble requires running all three models and averaging their outputs, tripling the inference time compared to a single XGBoost model.

3. Marginal improvement: The performance gain from the ensemble does not justify the increased complexity and operational cost. In production IDS deployments, the trade-off favors the simpler, faster single model.

9. Conclusion

This modelling phase comprehensively evaluated six machine learning algorithms for network intrusion detection on the CICIDS2017 dataset. The key findings are:

1. Non-linear models are essential: Logistic Regression's significantly lower performance (F1-macro: 0.567) compared to tree-based models (>0.78) confirms that network intrusion patterns require non-linear decision boundaries.

2. Gradient boosting outperforms bagging: XGBoost (0.859) and LightGBM (0.830) outperform Random Forest (0.827) and Extra Trees (0.813) in F1-macro, demonstrating that sequential error correction is more effective than parallel averaging for this imbalanced dataset.

3. Aggressive class rebalancing can backfire: Hyperparameter tuning with class_weight='balanced' degraded all three tuned models, with LightGBM collapsing to F1-macro 0.097. Extreme minority classes (Heartbleed: 9 train samples, SQL Injection: 17 train samples) make balanced weighting unstable.

4. Data limitation is the primary bottleneck: The remaining performance gap (particularly on Web Attack classes, F1 ~0.3) cannot be solved by model selection or tuning alone. Improving detection of these rare attacks would require either more labeled samples or additional engineered features capturing application-layer behavior.

5. Selected model: XGBoost (baseline) achieves the best balance of minority class detection (F1-macro: 0.859), overall accuracy (99.87%), and practical deployability. The trained model has been saved for deployment at models/xgboost_best_model.joblib.